

Guided Soft Actor–Critic

Blind Go2-W Locomotion — Theory and Implementation

Tamer Bora İkizoğlu

Istanbul Technical University

Outline

The problem

Guided SAC: the idea

Architecture

Objectives

Schedules

Data flow

Implementation

The problem

Blind locomotion is a POMDP

In simulation we can read anything: body linear velocity, a height scan of the terrain, contact forces, friction, every randomized dynamics parameter.

On the real robot almost none of that exists. The deployed policy sees only proprioception: joint encoders, IMU, its own previous action.

The asymmetry

Training-time information $\gg$ deployment-time information. A policy trained on what it *cannot* have at deployment is useless; a policy that ignores what the simulator knows is wasteful.

Guided SAC is one answer: let a *privileged* network help train a *blind* network that is the only thing shipped.

Two observation channels



Proprioceptive, noisy, 5-frame history

5×57 : base ang. vel. (3), projected gravity (3), velocity command (3), joint pos (16), joint vel (16), previous action (16)

Privileged, clean, single frame

$57 + 3 + 187$: same frame without noise, base *linear* velocity (3), height scan (187)

Action space: 16 dims = 12 leg position targets + 4 wheel velocity targets, tanh-squashed and scaled by per-dimension bounds (legs 5.0, wheels 10.0).

Guided SAC: the idea

Origin, and what we changed

Haklıdır & Temeltaş (2021) and Çerçi & Temeltaş (2024) train **two actors**: a *guiding actor* that sees clean state, and a *control actor* that sees degraded state and is pulled toward the guide's actions.

	Çerçi (2024)	This project
Why the control actor is handicapped	adversarial perturbation	<i>genuine</i> partial observability
Guiding actor sees	clean state s_t	privileged $\tilde{s}_t$ (height scan, lin. vel.)
Control actor sees	perturbed s_t^p	proprioception o_t + history
Shapes	identical	different (285 vs 532)
Agents	$k = 3, 5, 7$	$k = 1$ (single agent)

Two consequences: the paper's $s_t^p \rightarrow o_t$ mapping is *not* cosmetic (different dimensionality), and every multi-agent construction collapses at $k = 1$ — it must be re-derived, not copied.

Guided SAC is three mechanisms, not one

M1 — Asymmetric critic

$Q(\tilde{s}_t, a)$ sees everything,
 $\pi_f(a|o_t)$ does not

cost: zero extra networks
fixes credit assignment

M2 — Guide as behavior policy

the teacher also *acts*
in the environment

cost: one extra actor
widens state visitation

M3 — Action-space imitation

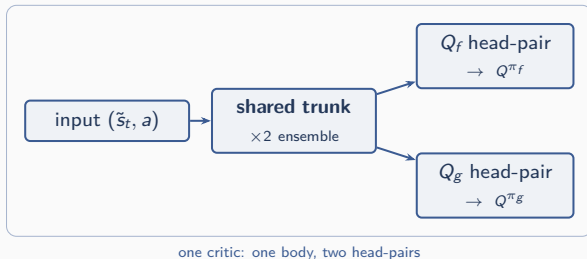
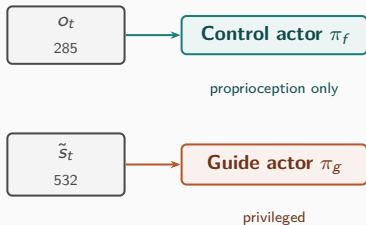
pull π_f toward π_g
with weight λ

cost: one norm term
dense low-variance target

The literature ships these **bundled**. Separating them matters: M1 is free and not specific to Guided SAC, while M2 is the expensive one. Our design keeps M1, keeps M3 in softened form, and uses only a *small* amount of M2 — for a different reason than the original (next slides).

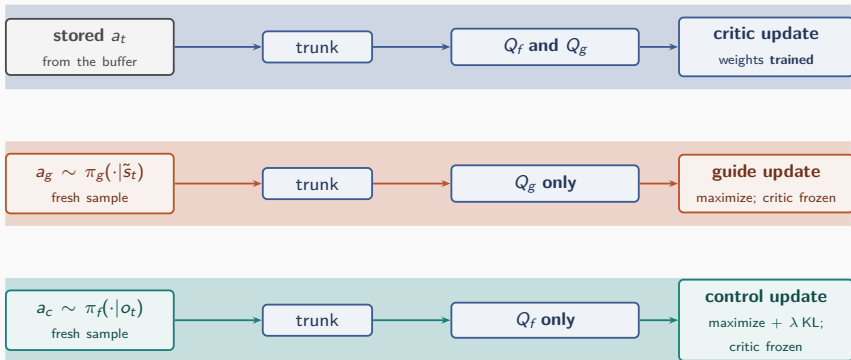
Architecture

Three networks



Two actors differing only in what they see, and one critic whose body is shared but whose output heads are split. **Which** action is fed to the critic — and which head-pair answers — depends on the update, and is the next slide.

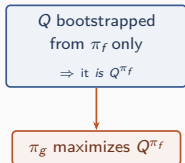
Which action reaches which head



The trunk is the *same* network in all three lanes. Two rules make the whole design readable: the critic is **only ever trained on actions that were actually executed**, and each actor **only ever reads its own head-pair** — so the student optimizes the value of the deployed policy and never the teacher's.

Why the guide needs its own value head

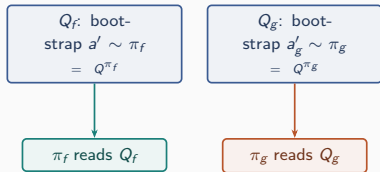
One critic (naive)



one greedy improvement
step over *the student*;
improvements never re-enter the
bootstrap $\Rightarrow$ they never compound

The teacher's advantage is **capped by construction** — contradicting the premise that a fully-observed teacher is meaningfully better.

Two head-pairs (ours)



Each actor optimizes against *its own* fixed point, so the teacher can run ahead. The student still optimizes Q^{π_f} — the **deployed** policy's value.

This recovers, in twin- Q form, the role of Çerçi's separate V network.

Objectives

Critic: one template, two instantiations

Both head-pairs are trained on the **same stored transition** $(\tilde{s}_t, a_t, r, \tilde{s}_{t+n})$ and differ only in the policy used to bootstrap:

$$y_{\bullet} = r_t^{(n)} + \gamma^n \left(z - \alpha_{\bullet} \log \pi_{\bullet}(a'_{\bullet} | \cdot) \right) (1 - d)$$

head-pair	bootstrap action	temperature	estimates
Q_f	$a' \sim \pi_f(\cdot o_{t+n})$	α_f	Q^{π_f} — the deployed policy
Q_g	$a'_g \sim \pi_g(\cdot \tilde{s}_{t+n})$	α_g	Q^{π_g} — the teacher

$$\mathcal{L}_Q = \text{CE}(\Pi[y_f] \parallel Q_{f,i}(\tilde{s}_t, a_t)) + \text{CE}(\Pi[y_g] \parallel Q_{g,i}(\tilde{s}_t, a_t)), \quad i \in \{1, 2\}$$

The critic is distributional: z are fixed bin values and Π projects the target back onto that support.
 n -step returns and the EMA target network come from the backbone unchanged.

Actors: one new term in the whole method

Guide — ordinary SAC on privileged input, against its own heads:

$$\mathcal{L}_{\pi_g} = \mathbb{E} \left[\alpha_g \log \pi_g(a_g | \tilde{s}_t) - \min_i Q_{g,i}(\tilde{s}_t, a_g) \right]$$

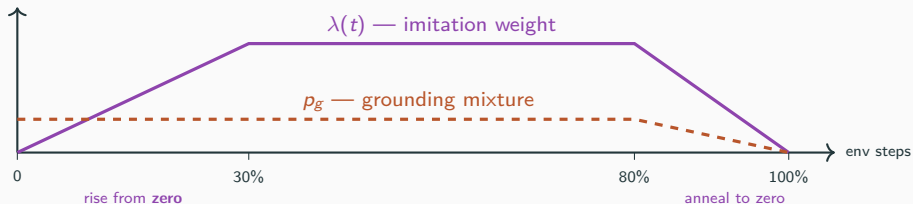
Control — the same objective on proprioception, plus guidance:

$$\mathcal{L}_{\pi_f} = \mathbb{E} \left[\alpha_f \log \pi_f(a_c | o_t) - \min_i Q_{f,i}(\tilde{s}_t, a_c) + \underbrace{\lambda(t) \bar{q} D_{\text{KL}}(\pi_f(\cdot | o_t) \parallel \text{sg}[\pi_g(\cdot | \tilde{s}_t)])}_{\text{the only term Guided SAC adds}} \right]$$

- **KL, not L_2 .** Both policies are tanh-Gaussian, so the KL is closed-form and matches the teacher's *variance*, not just its mean — the student may stay uncertain where the teacher is. The teacher is stop-graduated.
- $\bar{q} = \mathbb{E}[\min_i Q_{f,i}]$ keeps λ dimensionless as values grow; α_f, α_g are learned **separately**.

Schedules

Guidance is scheduled, grounding is not



λ **starts at 0** because an unconverged teacher emits noise — a constant weight injects it exactly when the student is most malleable.

λ **ends at 0** so the student finishes under its own constraints — the cheapest mitigation of the imitation gap.

p_g **is on from step 0** — not exploration but *value grounding*: without the guide ever acting, $(\tilde{s}_t, a_g)$ never enters the buffer and Q_g is queried off its own training distribution.

The imitation gap — the method's real weakness

π_g solves the *fully observed* MDP. π_f must solve the *POMDP*. These are different problems, and projecting one onto the other is not optimal in general.

Concretely, for blind locomotion

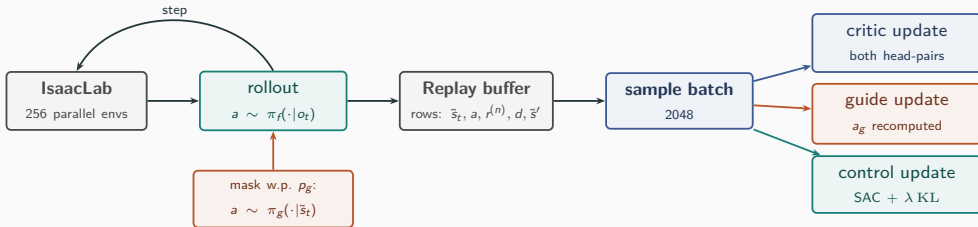
A teacher that sees the height scan never learns to probe the ground with a foot, never steps cautiously into the unknown, never moves to enrich its own IMU signal — it has no need to. Imitating it can therefore *suppress exactly the behaviours that keep a blind robot alive*.

Mitigations built in, weakest to strongest:

1. **KL rather than L_2** — the student may stay uncertain where the teacher is.
2. $\lambda \rightarrow 0$ — guidance is removed before training ends.
3. **Latent supervision** (planned): supervise a learned *representation* $\hat{z} = E(o_t)$ against privileged z and let the policy decide how to use it — closes the gap by construction.

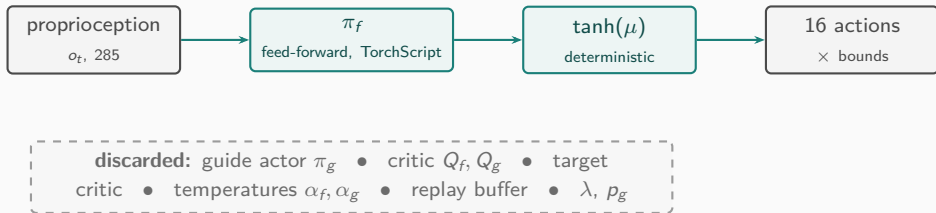
Data flow

Training loop



- a_g is **never stored**. It is recomputed from the sampled batch's own $\tilde{s}_t$ at update time, so teacher–state alignment is true by construction and the buffer schema is unchanged — no off-by-one is even representable.
- Mixed rows are stored **unmarked**: off-policy TD does not need provenance.

Deployment — what survives



- The deployed artifact is **architecturally identical** to a vanilla SAC actor. Guided training costs *nothing* at inference — the strongest practical argument for asymmetric methods.
- Nothing privileged can leak: the actor consumes a fixed 285-dim slice and the export path never reads the guide's files.

Implementation

Code map

Built as an **extension** of the FlashSAC backbone, never a fork: the vendored learning core is byte-identical, and without the `--guided` flag the pipeline is the unmodified baseline.

<code>gsac/config.py</code>	config dataclass; $\lambda(t)$, $p_g(t)$; fail-fast validation
<code>gsac/network.py</code>	GuidedDoubleCritic: shared trunks + second head-pair
<code>gsac/update.py</code>	the three losses above, each traced to its source equation
<code>gsac/agent.py</code>	GuidedFlashSACAgent: networks, rollout mixing, updates

<code>train.py --guided</code>	launcher flag; everything guided is behind it
--------------------------------	---

The subclass overrides `sample_actions` (mixing), `update` (the three losses) and `save/load` (guide checkpoints); everything else is inherited.

One codebase, four experiments

The mechanism decomposition is expressible as *configuration*, which is what makes an honest ablation cheap:

Rung	Isolates	Setting	Cost
A1	vanilla SAC	symmetric critic (override)	baseline
A2	M1 asymmetric critic	<code>--no-train_guide</code>	no guide built
A4	M2 grounding mixture	<code>--lambda_max 0</code>	guide trained, no imitation
A5	M3 full Guided SAC	default	everything on

The question this design exists to answer

Which of the three mechanisms actually pays — and is the second actor worth its cost? The literature has never separated them. Whatever the answer, it is informative.

Summary

- Blind locomotion is a genuine POMDP; the simulator knows far more than the robot ever will.
- Guided SAC bundles three separable mechanisms. We keep the free one (asymmetric critic), soften the risky one (imitation via scheduled KL), and use the expensive one sparingly and for a different reason than the original (value grounding).
- The single-agent reduction is not a copy: the teacher needs **its own value head**, otherwise its advantage is capped by construction.
- The imitation gap is the method's real theoretical weakness; latent supervision is the principled successor if action-space imitation does not pay.
- **Deployment cost is exactly zero** — the shipped network is a plain SAC actor on proprioception.